

Dynamic Memory Allocation in C

Dynamic Memory Allocation in C

Dynamic memory allocation is a crucial concept in programming, particularly in languages like C where memory management is manual. In C, the `malloc()` and `free()` functions provide a mechanism for allocating and deallocating memory during program execution.

- ▶ The `malloc()` function dynamically allocates a specified number of bytes in memory and returns a pointer to the beginning of the allocated memory block.
- ▶ Syntax: `void *malloc(size_t size);`
- ▶ `size` specifies the number of bytes to allocate. (`size_t`: data type to represent sizes of objects in memory; unsigned integer of system dependent size; usually, equivalent to unsigned long.)
- ▶ Returns a pointer of type `void *`, which can be cast to any pointer type.

Dynamic Memory Allocation in C

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr; int n = 5;
        // Allocate memory to store 5 integers
    ptr = (int*)malloc(n * sizeof(int));
    if (ptr == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

        // Use the allocated memory
    for (int i = 0; i < n; i++) {
        ptr[i] = i + 1;
        printf("%d ", ptr[i]);
    }

        // Free the allocated memory
    free(ptr);
    return 0;
}
```

Dynamic Memory Allocation in C (cont'd)

- ▶ The `free()` function deallocates memory previously allocated by `malloc()` or similar functions.
- ▶ Syntax: `void free(void *ptr);`
- ▶ `ptr` is the pointer to the memory block to be deallocated.

Memory Management Best Practices

- ▶ **Always Check for NULL:** After calling `malloc()`, always check if the returned pointer is `NULL` to ensure successful allocation.
- ▶ **Free Memory After Use:** Release dynamically allocated memory using `free()` once it's no longer needed to prevent memory leaks.
- ▶ **Do Not Free Memory Twice:** Attempting to free the same memory block twice leads to undefined behavior.
- ▶ **Avoid Memory Fragmentation:** Excessive allocation and deallocation of memory can lead to fragmentation, impacting performance. Consider techniques like memory pooling to mitigate fragmentation.

Memory Allocation for Processes

- ▶ Memory allocation for processes involves dividing memory into different regions:
 - ▶ Code Segment
 - ▶ Data Segment
 - ▶ BSS Segment
 - ▶ Heap
 - ▶ Stack
- ▶ These regions serve different purposes and have distinct characteristics.

Code Segment

- ▶ The code segment, also known as the text segment, stores the executable code of the program.
- ▶ Instructions in this segment are typically read-only and not modifiable during program execution.
- ▶ Example: Instructions for functions, loops, conditionals, etc.

Data Segment

- ▶ The data segment stores initialized data, including global and static variables.
- ▶ Contents of this segment are initialized explicitly by the programmer.
- ▶ Example: Global variables, static variables, constants.

BSS Segment

- ▶ The BSS segment, which stands for "Block Started by Symbol," stores uninitialized data.
- ▶ This segment is zero-initialized before the program starts execution.
- ▶ Example: Uninitialized global and static variables.

Process Stack

- ▶ The process stack is a region of memory allocated for:
 - ▶ Function calls
 - ▶ Local variables
 - ▶ Function call information
- ▶ It operates in a LIFO (Last-In, First-Out) manner.
- ▶ Automatic memory management by the compiler and CPU.
- ▶ Fixed size, typically determined at compile time.

Heap

- ▶ The heap is a region of memory allocated for dynamic memory allocation.
- ▶ Memory on the heap is managed manually by the programmer.
- ▶ Allows for dynamic allocation and deallocation of memory during runtime.
- ▶ Unlimited size, limited only by available system memory.
- ▶ Accessing memory on the heap is slower compared to the stack.

Static Variables

- ▶ Variables like `sp` and `val` in `stack.c`, and `buf` and `bufp` in `getch.c`, are private to their respective source files.
- ▶ `static` declaration for external variables or functions limits their scope to the source file being compiled.
- ▶ External `static` hides names, providing a way to keep them private.
- ▶ Example: `static char buf[BUFSIZE]; static int bufp = 0;`
- ▶ External `static` can be applied to functions, making their names invisible outside the file.

Static Variables Example

```
static char buf[BUFSIZE];  
static int bufp = 0;  
  
int getch(void) { ... }  
  
void ungetch(int c) { ... }
```

Internal Static Variables

- ▶ Internal static variables are local to a function but persist across function calls.
- ▶ They provide private, permanent storage within a single function.
- ▶ Example: `static int count = 0;`

Internal Static Variables Example

```
void exampleFunction() {  
    static int count = 0;  
    // count retains its value across calls  
    // other local variables are recreated each time  
}
```

Register Variables

- ▶ Register declaration advises the compiler that the variable will be heavily used.
- ▶ Intended for variables to be placed in machine registers for potential speedup.
- ▶ Limited number of variables in each function may be kept in registers.
- ▶ Specific restrictions on types and numbers of register variables vary across machines.
- ▶ Compiler may ignore the register advice.

Register Variables Example

```
register int x;  
register char c;  
  
f(register unsigned m, register long n) {  
    register int i;  
    // ...  
}
```

Block Structure

- ▶ C is a block-structured language in that code is organized into blocks or compound statements.
- ▶ Variables can be defined within blocks.
- ▶ Block-scoped variables hide outer block variables.
- ▶ Automatic variables in a block are re-initialized upon block entry.
- ▶ Automatic variables hide external variables and functions of the same name.

Block Structure Example

```
if (n > 0) {  
    int i; /* declare a new i */  
    for (i = 0; i < n; i++)  
        ...  
}
```

The scope of `i` is the “true” branch of the `if` ; this `i` is unrelated to any `i` outside the block.

Block Structure Example

```
int x;  
int y;  
  
f(double x)  
{  
    double y;  
}
```

Within `f`, occurrences of `x` refer to the parameter, which is a double; outside, they refer to the external `int`.

Initialization

- ▶ External and static variables are guaranteed to be initialized to zero if not explicitly initialized.
- ▶ Automatic and register variables have undefined initial values (garbage).
- ▶ Scalar variables may be initialized with definition.
- ▶ For external and static variables, the initializer must be a constant expression. Initialization is done once
- ▶ For automatic and register variables, any expression using previously defined values, even function calls can be the initializer.

Initialization Examples

```
int x = 1;
char squota = '\'';
long day = 1000L * 60L * 60L * 24L; /* milliseconds/day */

int days[] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };

char pattern[] = "ould";
           /* equivalent to {'o', 'u', 'l', 'd', '\0'} */
```

Unsorted Singly Linked List

Unsorted Singly Linked List

Introduction

```
#include <stdio.h>
#include <stdlib.h>

// Node structure for the linked list
typedef struct Node {
    int data;
    struct Node* next;
} Node;

// Function to create a new node
Node* createNode(int value) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Memory allocation failed\n");
        exit(1);
    }
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}
```